

```
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/constants/BofaJcrPropertyConstants.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/constants/BofaJcrPropertyConstants.java
index 6356e78..832a948 100644
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/constants/BofaJcrPropertyConstants.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/constants/BofaJcrPropertyConstants.java
@@ -14,5 +14,4 @@ public class BofaJcrPropertyConstants {
```

```
    public static final String PROP_ASSET_METADATA_PATHS = "assetMetadataPaths";
```

```
-
}
```

```
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/executor/JcrResolverExecutor.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/executor/JcrResolverExecutor.java
index 1383ce2..a3c0865 100644
```

```
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/executor/JcrResolverExecutor.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/executor/JcrResolverExecutor.java
@@ -2,6 +2,7 @@
package com.adobe.aem.acs.bofa.core.executor;
```

```
import com.adobe.aem.acs.bofa.core.constants.BofaSyncConstants;
+import org.apache.sling.api.resource.LoginException;
import org.apache.sling.api.resource.PersistenceException;
import org.apache.sling.api.resource.ResourceResolver;
import org.apache.sling.api.resource.ResourceResolverFactory;
@@ -46,10 +47,11 @@ public class JcrResolverExecutor {
```

```
    return result;
```

```
+    } catch (LoginException e) {
+        logger.error("Could not obtain service resolver: {}", e.getMessage(), e);
+    } catch (PersistenceException e) {
```

```
-
```

```
-        logger.warn(" Commit failed on attempt {}: {}", attempt, e.getMessage());
// Wait or backoff can be added here
+        logger.warn(" Commit failed on attempt {}: {}", attempt, e.getMessage());
+    }
```

```
    }
    throw new PersistenceException("JCR commit failed after " + MAX_RETRIES + " attempts");
@@ -67,7 +69,7 @@ public class JcrResolverExecutor {
    return work.execute(resolver);
```

```
    } catch (Exception e) {
-        logger.error("Æ Error in JCR unit of work (read-only)", e);
+        logger.error("Error in JCR unit of work (read-only)", e);
        throw e;
    }
}
```

```
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/listeners/IncomingReplicationListener.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/listeners/IncomingReplicationListener.java
index ec41c7e..fd655d0 100644
```

```
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/listeners/IncomingReplicationListener.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/listeners/IncomingReplicationListener.java
@@ -2,9 +2,8 @@ package com.adobe.aem.acs.bofa.core.listeners;
```

```

import com.adobe.aem.acs.bofa.core.constants.BofaSyncConstants;
import com.adobe.aem.acs.bofa.core.executor.JcrResolverExecutor;
-import com.adobe.aem.acs.bofa.core.models.ContentFragmentInfo;
import com.adobe.aem.acs.bofa.core.services.*;
-import com.drew.lang.annotations.NotNull;
+import org.apache.sling.api.resource.Resource;
import org.apache.sling.api.resource.ResourceResolver;
import org.apache.sling.api.resource.observation.ResourceChange;
import org.apache.sling.api.resource.observation.ResourceChangeListener;
@@ -13,7 +12,6 @@ import org.osgi.service.component.annotations.Reference;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

-import java.util.ArrayList;
import java.util.List;

/**
@@ -25,7 +23,7 @@ import java.util.List;
    property = {
        ResourceChangeListener.PATHS + "=" + BofaSyncConstants.BOFA_SYNC_MERGE_PATH,
        ResourceChangeListener.CHANGES + "=ADDED",
-//        ResourceChangeListener.CHANGES + "=CHANGED",
+        ResourceChangeListener.CHANGES + "=CHANGED",
        ResourceChangeListener.PROPERTY_NAMES_HINT + "=cq:lastReplicated"
    }
)
@@ -39,9 +37,6 @@ public class IncomingReplicationListener implements ResourceChangeListener {
    @Reference
    private MergeConfigService mergeConfigService;

-//    @Reference
-//    private ContentFragmentService contentFragmentService;
-
    @Reference
    private ContentFragmentMergeWorkflowService contentFragmentMergeWorkflowService;

@@ -57,34 +52,23 @@ public class IncomingReplicationListener implements ResourceChangeListener {
    try {
        executor.runWithWriteAccess((ResourceResolver resourceResolver) -> {
-            List<ContentFragmentInfo> incomingContentFragments = new ArrayList<>();

            for (ResourceChange change : changes) {
                String path = change.getPath();
                ResourceChange.ChangeType type = change.getType();

-//                logger.info("Replicated content change detected â†’ path: {}, changeType: {}, mergeTempPath: {}",
-//                    path, type, mergeConfigService.getIncomingMergePath());
+                logger.info("Replicated content change detected path: {}, changeType: {}, mergeTempPath: {}", path,
+                    type, mergeConfigService.getIncomingMergePath());
                if (path.startsWith(mergeConfigService.getIncomingMergePath())) {
+                    Resource resource = resourceResolver.getResource(path);
+                    if (resource != null) {

```

```

+
+         assetService.checkNode(resourceResolver, path);
-
-         assetService.checkNode(resourceResolver, path);
-/*
-         ContentFragmentInfo incomingContentFragment =
contentFragmentService.checkNode(resourceResolver, path);
-         if(incomingContentFragment != null){
-             if(incomingContentFragment.isMissingContentFragmentModel()) {
-                 contentFragmentMergeWorkflowService.addContentFragmentMissingModel(resourceResolver,
incomingContentFragment);
-             } else {
-                 incomingContentFragments.add(incomingContentFragment);
-             }
-         }
-     }
- }

-     logger.info("Starting merge with {} content fragments", incomingContentFragments.size());
-     contentFragmentMergeWorkflowService.startContentFragmentMerge(resourceResolver,
incomingContentFragments);
-*/
-
-     }
+
+     logger.info("Starting merge with content fragments");
+     contentFragmentMergeWorkflowService.startContentFragmentMerge(resourceResolver);

```

```

diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/AssetService.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/AssetService.java
index ecb2987..5b29a1c 100644

```

```

--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/AssetService.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/AssetService.java
@@ -1,6 +1,7 @@
package com.adobe.aem.acs.bofa.core.services;

```

```

import com.adobe.aem.acs.bofa.core.models.AssetMergeMetadata;
+import com.day.cq.dam.api.Asset;
import org.apache.sling.api.resource.ResourceResolver;

```

```

import java.util.List;
@@ -13,4 +14,8 @@ public interface AssetService {

```

```

    List<AssetMergeMetadata> getAssetMergeMetadataList(ResourceResolver resourceResolver);

```

```

+    Asset createAssetFromSource(ResourceResolver resourceResolver, Asset source, String assetPath);
+
+    Asset updateAssetFromSource(ResourceResolver resourceResolver, String assetPath);
+
+}

```

```

diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/ContentFragmentMergeWorkflowService.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/ContentFragmentMergeWorkflowService.java
index 0bfed8d..2b2c53d 100644

```

```

--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/ContentFragmentMergeWorkflowService.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/ContentFragmentMergeWorkflowService.java
@@ -13,8 +13,8 @@ public interface ContentFragmentMergeWorkflowService {

    void removeContentFragmentMissingModel(ResourceResolver resourceResolver, ContentFragmentInfo
contentFragmentInfo);

-    void closeContentFragmentMerge(ResourceResolver resourceResolver, String contentFragmentPath, String
variationName);
+    void closeContentFragmentVariationMerge(ResourceResolver resourceResolver, String contentFragmentPath,
String variationName);

-    void endContentFragmentMerge(ResourceResolver resourceResolver);
+    void closeContentFragmentMerge(ResourceResolver resourceResolver, String contentFragmentPath);

}
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/MergeStatusService.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/MergeStatusService.java
new file mode 100644
index 00000000..9e92b6a
--- /dev/null
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/MergeStatusService.java
@@ -0,0 +1,10 @@
+package com.adobe.aem.acs.bofa.core.services;
+
+import java.util.List;
+import java.util.Map;
+
+public interface MergeStatusService {
+
+    Map<String, List<String>> getMergeStatus();
+
+}
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetMergeServiceImpl.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetMergeServiceImpl.java
index 0989bf9..7e91648 100644
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetMergeServiceImpl.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetMergeServiceImpl.java
@@ -6,6 +6,7 @@ import com.adobe.aem.acs.bofa.core.services.*;
import com.adobe.aem.acs.bofa.core.util.BofaAssetUtils;
import com.adobe.aem.acs.bofa.core.util.BofaJcrUtils;
import com.day.cq.dam.api.Asset;
+import com.day.cq.dam.api.AssetManager;
import com.day.crx.JcrConstants;
import org.apache.commons.lang3.StringUtils;
import org.apache.sling.api.resource.*;
@@ -15,6 +16,7 @@ import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import javax.jcr.RepositoryException;
+import java.io.InputStream;
import java.text.SimpleDateFormat;
import java.util.*;

```

```

@@@ -31,6 +33,9 @@@ public class AssetMergeServiceImpl implements AssetMergeService {
    @Reference
    private AssetService assetService;

+    @Reference
+    private AssetMergeWorkflowService assetMergeWorkflowService;
+
    @Reference
    private MergeConfigService mergeConfigService;

@@@ -73,21 +78,22 @@@ public class AssetMergeServiceImpl implements AssetMergeService {

    ModifiableValueMap incomingAssetNodeJcrContent =
Objects.requireNonNull(resourceResolver.getResource(assetMergeMetadata.getAssetTempPath() + "/" +
JcrConstants.JCR_CONTENT)).adaptTo(ModifiableValueMap.class);

+    Asset source =
Objects.requireNonNull(Objects.requireNonNull(resourceResolver.getResource(assetMergeMetadata.getAssetTempPath() + "/" +
JcrConstants.JCR_CONTENT)).adaptTo(Asset.class));
    Resource destinationAssetResource = resourceResolver.getResource(assetMergeMetadata.getAssetPath());
-    Resource sourceAssetResource = resourceResolver.getResource(assetMergeMetadata.getAssetTempPath());
+
+    boolean isNewAsset;

    if(destinationAssetResource == null) {
-        // asset doesn't exist on destination author, copy to new location
-//        Session session = resourceResolver.adaptTo(Session.class);
-//        Objects.requireNonNull(session).getWorkspace().copy(assetMergeMetadata.getAssetTempPath(),
assetMergeMetadata.getAssetPath());
-        // add the new path to list of assets with merge conflicts
-        existingAssetSet.add(assetMergeMetadata.getAssetPath());
+        isNewAsset = true;
+        assetService.createAssetFromSource(resourceResolver, source, assetMergeMetadata.getAssetPath());

    } else {
-        Asset sourceAsset = sourceAssetResource.adaptTo(Asset.class);
-        Asset destinationAsset = destinationAssetResource.adaptTo(Asset.class);
-        String sourceSha1 = BofaAssetUtils.getSha1(sourceAsset);
-        String destinationSha1 = BofaAssetUtils.getSha1(destinationAsset);
+        logger.info("Updating asset: {}", assetMergeMetadata.getAssetPath());
+        isNewAsset = false;

+        Asset destination = destinationAssetResource.adaptTo(Asset.class);
+        String sourceSha1 = BofaAssetUtils.getSha1(source);
+        String destinationSha1 = BofaAssetUtils.getSha1(destination);
        logger.info("sourceSha1: {} destinationSha1: {}", sourceSha1, destinationSha1);

        boolean hasMergeConflicts = !StringUtils.equals(sourceSha1, destinationSha1);
@@@ -100,20 +106,24 @@@ public class AssetMergeServiceImpl implements AssetMergeService {
    logger.info("Asset hasMergeConflicts: {}", hasMergeConflicts);
    assetMetadata.put(BofaJcrUtils.getNamespacedProperty("hasMergeConflict"), hasMergeConflicts);

-    incomingAssetNodeJcrContent.put(BofaJcrUtils.getNamespacedProperty("sha1"), sourceSha1);

```

```

        // add the new path to list of assets with merge conflicts
        existingAssetSet.add(assetMergeMetadata.getAssetPath());
    }
+    incomingAssetNodeJcrContent.put(BofaJcrUtils.getNamespacedProperty("sha1"), sourceSha1);

-    MetadataMerge metadataMerge = metadataMergeService.getMetadataMerge(resourceResolver,
getMetadataMergeMetadata(assetMergeMetadata));
-    logger.info("metadataMerge isContentIdentical: {}", metadataMerge);
-    if(!metadataMerge.isContentIdentical()){
-        existingAssetMetadataSet.add(assetMergeMetadata.getAssetPath());
-        incomingAssetNodeJcrContent.put(BofaJcrUtils.getNamespacedProperty("mergeMetadataStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_OPEN);

-    }
+    }
+    MetadataMerge metadataMerge = metadataMergeService.getMetadataMerge(resourceResolver,
getMetadataMergeMetadata(assetMergeMetadata));
+    logger.info("metadataMerge isContentIdentical: {}", metadataMerge);
+    if(!metadataMerge.isContentIdentical()){
+        existingAssetMetadataSet.add(assetMergeMetadata.getAssetPath());
+        incomingAssetNodeJcrContent.put(BofaJcrUtils.getNamespacedProperty("mergeMetadataStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_OPEN);
+    }

+    if(isNewAsset && metadataMerge.isContentIdentical()){
+        assetMergeWorkflowService.closeAssetMerge(resourceResolver, assetMergeMetadata.getAssetPath());
    }

```

```

@@@ -267,7 +277,9 @@@ public class AssetMergeServiceImpl implements AssetMergeService {
}

```

```

    private String getLastModifiedDate(ResourceResolver resourceResolver, Asset asset) {
-
+    if(asset == null){
+        return "N/A";
+    }
    Resource assetResource = resourceResolver.getResource(asset.getPath());
    Resource originalResource =
Objects.requireNonNull(assetResource).getChild("jcr:content/renditions/original/jcr:content");

```

```

diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetMergeWorkflowServiceImpl.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetMergeWorkflowServiceImpl.java
index 3c3d7cd..b8e74e9 100644
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetMergeWorkflowServiceImpl.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetMergeWorkflowServiceImpl.java
@@@ -16,6 +16,8 @@@ import org.slf4j.LoggerFactory;

```

```

import java.util.Objects;

+import static com.adobe.aem.acs.bofa.core.constants.BofaJcrPropertyConstants.PROP_ASSET_PATHS;
+
@Component(service = AssetMergeWorkflowService.class, immediate = true)

```

```
public class AssetMergeWorkflowServiceImpl implements AssetMergeWorkflowService {
```

```
@@ -25,9 +27,6 @@ public class AssetMergeWorkflowServiceImpl implements AssetMergeWorkflowService
    @Reference
    private AssetService assetService;
```

```
-    @Reference
-    private AssetMergeService assetMergeService;
```

```
-
```

```
    public void closeAssetMerge(ResourceResolver resourceResolver, String assetPath){
        logger.debug("Closing Asset Merge assetPath: {}", assetPath);
```

```
@@ -36,15 +35,17 @@ public class AssetMergeWorkflowServiceImpl implements AssetMergeWorkflowService
    String assetJcrContentPath = assetMergeMetadata.getAssetTempPath() + "/" + JcrConstants.JCR_CONTENT;
    ModifiableValueMap assetJcrContent =
Objects.requireNonNull(resourceResolver.getResource(assetJcrContentPath)).adaptTo(ModifiableValueMap.class);
```

```
-    AssetMerge assetMerge = assetMergeService.getAssetMerge(resourceResolver, assetMergeMetadata);
+    logger.info("Close asset merge for asset: {}", assetMergeMetadata.getAssetPath());
+    assetJcrContent.put(BofaJcrUtils.getNamespacedProperty("mergeStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_CLOSED);
```

```
-    if(assetMerge.isInputStreamsEqual()){
-        logger.info("Close asset merge for asset: {}", assetMergeMetadata.getAssetPath());
-        assetJcrContent.put(BofaJcrUtils.getNamespacedProperty("mergeStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_CLOSED);
-    } else {
-        throw new IllegalStateException("Cannot close asset merge, content is not the same");
-    }
```

```
+    String mergeJcrContentPath = BofaSyncConstants.BOFA_SYNC_MERGE_PATH + "/" +
JcrConstants.JCR_CONTENT;
```

```
+    ModifiableValueMap mergeProperties =
Objects.requireNonNull(resourceResolver.getResource(mergeJcrContentPath)).adaptTo(ModifiableValueMap.class);
+
+    BofaJcrUtils.removePathFromProperty(mergeProperties, assetMergeMetadata.getAssetPath(),
PROP_ASSET_PATHS);
```

```
+    if (!BofaJcrUtils.isAssetInUse(mergeProperties, assetMergeMetadata.getAssetPath())) {
+        BofaJcrUtils.removeNode(resourceResolver, mergeProperties, assetMergeMetadata.getAssetPath(),
assetMergeMetadata.getAssetTempPath());
+    }

    }
```

```
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetServiceImpl.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetServiceImpl.java
index d4a7862..f37a7bc 100644
```

```
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetServiceImpl.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/AssetServiceImpl.java
@@ -9,6 +9,7 @@ import com.adobe.aem.acs.bofa.core.services.ContentFragmentService;
import com.adobe.aem.acs.bofa.core.services.MergeConfigService;
import com.adobe.aem.acs.bofa.core.util.BofaJcrUtils;
import com.day.cq.dam.api.Asset;
+import com.day.cq.dam.api.AssetManager;
```

```
-import javax.jcr.Node;
-import javax.jcr.RepositoryException;
-import javax.jcr.Value;
+import javax.jcr.*;
+import java.io.InputStream;
import java.util.*;
```

```

+ public void checkNode(ResourceResolver resourceResolver, String path){
+     logger.info("checkNode: {}", path);
+
+     if(StringUtils.endsWith(path, JcrConstants.JCR_CONTENT)){
+         ValueMap jcrValueMap = resourceResolver.getResource(path).adaptTo(ValueMap.class);
+         if(jcrValueMap != null){
+             if(jcrValueMap.containsKey(JcrConstants.JCR_PRIMARYTYPE)){
+                 String jcrPrimaryType = jcrValueMap.get(JcrConstants.JCR_PRIMARYTYPE, String.class);
+                 if(StringUtils.equals(jcrPrimaryType, "dam:AssetContent")){
+                     path = StringUtils.removeEnd(path, "/" + JcrConstants.JCR_CONTENT);
+                     logger.info("checkNode updated path: {}", path);
+                 }
+             }
+         }
+     }
+
+     Resource resource = Objects.requireNonNull(resourceResolver.getResource(path));
+     Asset asset = resource.adaptTo(Asset.class);
+     try {
+
+@@@ -121,6 +136,50 @@@ public class AssetServiceImpl implements AssetService {
+     }

```

.0.11-changes-full.txt[7/18/25, 11:48:50 AM]

```

+
+ public Asset updateAssetFromSource(ResourceResolver resourceResolver, String assetPath){
+
+     logger.info("Updating asset from source: {}", assetPath);
+
+     try {
+         Node node =
Objects.requireNonNull(resourceResolver.getResource(mergeConfigService.getIncomingMergePath() + "/" +
assetPath)).adaptTo(Node.class);
+         AssetMergeMetadata assetMergeMetadata = getAssetMergeMetadata(Objects.requireNonNull(node));
+
+         Asset source =
Objects.requireNonNull(resourceResolver.getResource(assetMergeMetadata.getAssetTempPath())).adaptTo(Asset.class
);
+
+         AssetManager assetManager = resourceResolver.adaptTo(AssetManager.class);
+
+         InputStream sourceStream = Objects.requireNonNull(source).getOriginal().getStream();
+         Session session = resourceResolver.adaptTo(Session.class);
+         ValueFactory valueFactory = Objects.requireNonNull(session).getValueFactory();
+         Binary binary = valueFactory.createBinary(sourceStream);
+
+         Asset asset = Objects.requireNonNull(assetManager).createOrUpdateAsset(assetPath, binary,
source.getMimeType(), false);
+
+         return asset;
+
+     } catch (RepositoryException e) {
+         throw new RuntimeException(e);
+     }
+
+ }
+
+
+
+
+
+
+
+
+

```

```

private void addIncomingPath(ResourceResolver resourceResolver, String path, boolean isContentFragment) throws
RepositoryException {

```

```

diff --git
a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/ContentFragmentMergeWorkflowServiceImpl.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/ContentFragmentMergeWorkflowServiceImpl.java
index b9e6ab7..608f4c4 100644
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/ContentFragmentMergeWorkflowServiceImpl.java
+++
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/ContentFragmentMergeWorkflowServiceImpl.java
@@ -17,11 +17,13 @@ import java.util.*;

```

```

import static com.adobe.aem.acs.bofa.core.constants.BofaJcrPropertyConstants.*;

```

```

-@Component(service = {ContentFragmentMergeWorkflowService.class}, immediate = true)
+@Component(service = ContentFragmentMergeWorkflowService.class, immediate = true)

```

```

public class ContentFragmentMergeWorkflowServiceImpl implements ContentFragmentMergeWorkflowService {

    private static final Logger logger = LoggerFactory.getLogger(ContentFragmentMergeWorkflowServiceImpl.class);

+   private final static String MASTER = "master";
+

    @Reference
    private ContentFragmentService contentFragmentService;
@@ -145,6 +147,12 @@ public class ContentFragmentMergeWorkflowServiceImpl implements ContentFragmentM
        properties.put(BofaJcrUtils.getNamespacedProperty(PROP_ASSET_METADATA_PATHS),
            updatedMetadataPaths.toArray(new String[0]));

+
+   // attempt to close any contrent fragments that have no changes
+   for(ContentFragmentInfo contentFragmentInfo : contentFragmentInfoList){
+       closeContentFragmentMerge(resourceResolver, contentFragmentInfo.getContentFragmentPath());
+   }
+
+   } catch (RepositoryException e) {
+       throw new RuntimeException(e);
+   }
@@ -203,61 +211,81 @@ public class ContentFragmentMergeWorkflowServiceImpl implements ContentFragmentM
        properties.put(BofaJcrUtils.getNamespacedProperty(PROP_CONTENT_FRAGMENT_MISSING_MODEL_PATHS),
            contentFragmentMissingModelPaths.toArray(new String[0]));
        logger.info("in here resolver.hasChanges: {}", resourceResolver.hasChanges());

-//     properties.remove("");
-
-   }

+   public void closeContentFragmentMerge(ResourceResolver resourceResolver, String contentFragmentPath) {
+       logger.info("Attempting to Close Content Fragment Variation Merge contentFragmentPath: {}",
            contentFragmentPath);

-   public void closeContentFragmentMerge(ResourceResolver resourceResolver, String contentFragmentPath, String
            variationName){
-       logger.debug("Closing Content Fragment Variation Merge variationName: {}, contentFragmentPath: {}",
            variationName, contentFragmentPath);
+       ContentFragmentInfo contentFragmentInfo =
            contentFragmentService.getContentFragmentInfo(resourceResolver, contentFragmentPath);

-       try {
+       String contentFragmentJcrContentPath = contentFragmentInfo.getContentFragmentTempPath() + "/" +
            JcrConstants.JCR_CONTENT;
+       Resource contentFragmentJcrContent =
            Objects.requireNonNull(resourceResolver.getResource(contentFragmentJcrContentPath));
+       logger.info("contentFragmentJcrContentPath: {}, {}", contentFragmentJcrContentPath,
            contentFragmentJcrContent);
+       boolean canCloseMerge = true;

```

```

-        ContentFragmentInfo contentFragmentInfo =
contentFragmentManagerService.getContentFragmentInfo(resourceResolver, contentFragmentPath);
+        Resource dataResource = Objects.requireNonNull(contentFragmentJcrContent).getChild("data");
+        for (Resource child : Objects.requireNonNull(dataResource).getChildren()) {
+            ModifiableValueMap properties = Objects.requireNonNull(child).adaptTo(ModifiableValueMap.class);
+            String status = Objects.requireNonNull(properties).get(BofaJcrUtils.getNamespacedProperty("mergeStatus"),
String.class);
+
if(Objects.requireNonNull(status).equals(BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_OPEN)){
+            canCloseMerge = false;
+        }
+    }

-        String contentFragmentJcrContentPath = contentFragmentInfo.getContentFragmentTempPath() + "/" +
JcrConstants.JCR_CONTENT;
-        Node contentFragmentJcrContent =
Objects.requireNonNull(resourceResolver.getResource(contentFragmentJcrContentPath)).adaptTo(Node.class);
-        assert contentFragmentJcrContent != null;
+        if(canCloseMerge){
+            logger.info("Closing Content Fragment Merge contentFragmentPath: {}", contentFragmentPath);

-        ContentFragmentMerge contentFragmentMerge =
contentFragmentManagerService.getContentFragmentMerge(resourceResolver, contentFragmentInfo);
+        ModifiableValueMap contentFragmentProperties =
Objects.requireNonNull(contentFragmentJcrContent).adaptTo(ModifiableValueMap.class);
+        Objects.requireNonNull(contentFragmentProperties).put(BofaJcrUtils.getNamespacedProperty("mergeStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_CLOSED);

-        ContentFragmentVariation variation = contentFragmentMerge.getVariation(variationName);
-        if(variation.isContentIdentical()){
-            logger.info("Close content fragment merge for variation: {} content fragment: {}", variation.getName(),
contentFragmentJcrContent.getPath());
-            Node variationNode = contentFragmentJcrContent.getNode("data").getNode(variation.getName());
-            variationNode.setProperty(BofaJcrUtils.getNamespacedProperty("mergeStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_CLOSED);
-        } else {
-            throw new IllegalStateException("Cannot close content fragment merge, content is not the same");
+            String mergeJcrContentPath = BofaSyncConstants.BOFA_SYNC_MERGE_PATH + "/" +
JcrConstants.JCR_CONTENT;
+            ModifiableValueMap mergeProperties =
Objects.requireNonNull(resourceResolver.getResource(mergeJcrContentPath)).adaptTo(ModifiableValueMap.class);
+
+            BofaJcrUtils.removePathFromProperty(mergeProperties, contentFragmentInfo.getContentFragmentPath(),
PROP_CONTENT_FRAGMENT_PATHS);
+
+            if(!BofaJcrUtils.isContentFragmentInUse(mergeProperties, contentFragmentPath)) {
+                BofaJcrUtils.removeNode(resourceResolver, mergeProperties,
contentFragmentInfo.getContentFragmentPath(), contentFragmentInfo.getContentFragmentTempPath());
+            }

-        } catch (RepositoryException e) {
-            throw new RuntimeException(e);
+

```

```

+     } else {
+         logger.info("Cannot Close Content Fragment Merge, check variations contentFragmentPath: {}",
contentFragmentPath);
+     }

+//         Node variationNode = contentFragmentJcrContent.getNode("data").getNode(variation.getName());
+//         variationNode.setProperty(BofaJcrUtils.getNamespacedProperty("mergeStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_CLOSED);
+
+
+     }

- public void endContentFragmentMerge(ResourceResolver resourceResolver){
-     logger.info("End content fragment merge workflow");
+ public void closeContentFragmentVariationMerge(ResourceResolver resourceResolver, String
contentFragmentPath, String variationName){
+     logger.info("Closing Content Fragment Variation Merge variationName: {}, contentFragmentPath: {}",
variationName, contentFragmentPath);

+     try {
-         String mergeJcrContentPath = BofaSyncConstants.BOFA_SYNC_MERGE_PATH + "/" +
JcrConstants.JCR_CONTENT;
-         Node mergeJcrContent =
Objects.requireNonNull(resourceResolver.getResource(mergeJcrContentPath)).adaptTo(Node.class);
-         mergeJcrContent.setProperty(BofaJcrUtils.getNamespacedProperty("status"),
BofaSyncConstants.BOFA_SYNC_WORKFLOW_CLOSED);
-
mergeJcrContent.setProperty(BofaJcrUtils.getNamespacedProperty(PROP_CONTENT_FRAGMENT_PATHS), new
String[] {});

-         Node tmpPath =
Objects.requireNonNull(resourceResolver.getResource(mergeConfigService.getIncomingMergePath())).adaptTo(Node.
class);
-         for (NodeIterator it = tmpPath.getNodes(); it.hasNext(); ) {
-             Node child = (Node) it.next();
-             child.remove();
-         }
+         ContentFragmentInfo contentFragmentInfo =
contentFragmentService.getContentFragmentInfo(resourceResolver, contentFragmentPath);
+
+         String contentFragmentJcrContentPath = contentFragmentInfo.getContentFragmentTempPath() + "/" +
JcrConstants.JCR_CONTENT;
+         Node contentFragmentJcrContent =
Objects.requireNonNull(resourceResolver.getResource(contentFragmentJcrContentPath)).adaptTo(Node.class);
+         assert contentFragmentJcrContent != null;
+
+         ContentFragmentMerge contentFragmentMerge =
contentFragmentMergeService.getContentFragmentMerge(resourceResolver, contentFragmentInfo);

+         ContentFragmentVariation variation = contentFragmentMerge.getVariation(variationName);
+         logger.info("Close content fragment merge for variation: {} content fragment: {}", variation.getName(),
contentFragmentJcrContent.getPath());
+         Node variationNode = contentFragmentJcrContent.getNode("data").getNode(variation.getName());

```

```

+         variationNode.setProperty(BofaJcrUtils.getNamespacedProperty("mergeStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_CLOSED);
+
+         if (variation.getName().equals(MASTER)){
+             closeContentFragmentMerge(resourceResolver, contentFragmentPath);
+         }
    } catch (RepositoryException e) {
        throw new RuntimeException(e);
    }
+
+    }
}
}
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/JavaDiffUtilDiffServiceImpl.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/JavaDiffUtilDiffServiceImpl.java
index fe72b73..2d39576 100644
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/JavaDiffUtilDiffServiceImpl.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/JavaDiffUtilDiffServiceImpl.java
@@ -39,9 +39,12 @@ public class JavaDiffUtilDiffServiceImpl implements JavaDiffUtilDiffService {
    String destination = contentFragmentDiffField.getDestination();

    if(contentFragmentDiffField.getContentType().equals("text/html")){
-        source = replacePTag(source);
-        destination = replacePTag(destination);
-
+        if(StringUtils.endsWith(source, "</p>")) {
+            source = replacePTag(source);
+        }
+        if(StringUtils.endsWith(destination, "</p>")) {
+            destination = replacePTag(destination);
+        }
    }

@@ -58,7 +61,9 @@ public class JavaDiffUtilDiffServiceImpl implements JavaDiffUtilDiffService {
    result = replaceMarkdown(result, "~", "del");

    if(contentFragmentDiffField.getContentType().equals("text/html")){
-        result = String.format("<p>%s</p>", result);
+        if(StringUtils.endsWith(source, "</p>")) {
+            result = String.format("<p>%s</p>", result);
+        }
    }
}

diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/MergeStatusServiceImpl.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/MergeStatusServiceImpl.java
new file mode 100644
index 0000000..809663b
--- /dev/null
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/MergeStatusServiceImpl.java
@@ -0,0 +1,118 @@
+package com.adobe.aem.acs.bofa.core.services.impl;

```

```

+
+import com.adobe.aem.acs.bofa.core.executor.JcrResolverExecutor;
+import com.adobe.aem.acs.bofa.core.models.*;
+import com.adobe.aem.acs.bofa.core.services.*;
+import org.apache.sling.api.resource.ResourceResolver;
+import org.osgi.service.component.annotations.Component;
+import org.osgi.service.component.annotations.Reference;
+import org.slf4j.Logger;
+import org.slf4j.LoggerFactory;
+
+import java.util.*;
+
+@Component(service = MergeStatusService.class, immediate = true)
+public class MergeStatusServiceImpl implements MergeStatusService {
+
+
+    private static final Logger logger = LoggerFactory.getLogger(MergeStatusServiceImpl.class);
+
+    @Reference
+    private JcrResolverExecutor executor;
+
+    @Reference
+    private ContentFragmentService contentFragmentService;
+
+    @Reference
+    private ContentFragmentMergeService contentFragmentMergeService;
+
+    @Reference
+    private MetadataService metadataService;
+
+    @Reference
+    private MetadataMergeService metadataMergeService;
+
+    @Reference
+    private AssetService assetService;
+
+    @Reference
+    private AssetMergeService assetMergeService;
+
+    public Map<String, List<String>> getMergeStatus(){
+        logger.info("Getting merge status paths");
+        Map<String, List<String>> mergeStatus = new HashMap<>();
+        try {
+            executor.runReadOnly((ResourceResolver resourceResolver) -> {
+                List<ContentFragmentInfo> contentFragmentInfoList =
contentFragmentService.getContentFragmentInfoList(resourceResolver);
+                List<ContentFragmentMerge> contentFragmentMergeList =
contentFragmentMergeService.getContentFragmentMergeList(resourceResolver, contentFragmentInfoList);
+
+                List<MetadataMergeMetadata> metadataMergeMetadataList =
metadataService.getMetadataMergeMetadataList(resourceResolver);
+                List<MetadataMerge> metadataMergeList =
metadataMergeService.getMetadataMergeList(resourceResolver, metadataMergeMetadataList);
+

```

```

+         List<AssetMergeMetadata> assetMergeMetadataList =
+         assetService.getAssetMergeMetadataList(resourceResolver);
+
+
+         Set<String> autoMerge = new HashSet<>();
+         Set<String> manualMerge = new HashSet<>();
+         Set<String> contentFragmentAutoMerge = new HashSet<>();
+         Set<String> contentFragmentManualMerge = new HashSet<>();
+         Set<String> metadataFragmentAutoMerge = new HashSet<>();
+         Set<String> metadataManualMerge = new HashSet<>();
+         Set<String> assetMerge = new HashSet<>();
+
+
+         for (ContentFragmentMerge contentFragmentMerge : contentFragmentMergeList) {
+             boolean isAutoMerge = true;
+             for (ContentFragmentVariation contentFragmentVariation : contentFragmentMerge.getVariations()) {
+                 if (!contentFragmentVariation.isAutoMerge()) {
+                     isAutoMerge = false;
+                 }
+             }
+
+             String contentFragmentPath =
+ contentFragmentMerge.getContentFragmentInfo().getContentFragmentPath();
+             if (isAutoMerge) {
+                 autoMerge.add(contentFragmentPath);
+                 contentFragmentAutoMerge.add(contentFragmentPath);
+             } else {
+                 manualMerge.add(contentFragmentPath);
+                 contentFragmentManualMerge.add(contentFragmentPath);
+             }
+         }
+
+         for (MetadataMerge metadataMerge : metadataMergeList) {
+             String assetPath = metadataMerge.getMetadataMergeMetadata().getAssetPath();
+             if (metadataMerge.isAutoMerge()) {
+                 autoMerge.add(assetPath);
+                 metadataFragmentAutoMerge.add(assetPath);
+             } else {
+                 manualMerge.add(assetPath);
+                 metadataManualMerge.add(assetPath);
+             }
+         }
+
+         for (AssetMergeMetadata assetMergeMetadata : assetMergeMetadataList) {
+             String assetPath = assetMergeMetadata.getAssetPath();
+             assetMerge.add(assetPath);
+             manualMerge.add(assetPath);
+         }
+
+         mergeStatus.put("contentFragmentAutoMerge", new ArrayList<>(contentFragmentAutoMerge));
+         mergeStatus.put("contentFragmentManualMerge", new ArrayList<>(contentFragmentManualMerge));
+         mergeStatus.put("metadataFragmentAutoMerge", new ArrayList<>(metadataFragmentAutoMerge));
+         mergeStatus.put("metadataManualMerge", new ArrayList<>(metadataManualMerge));
+         mergeStatus.put("assetMerge", new ArrayList<>(assetMerge));

```

```

+
+     mergeStatus.put("autoMerge", new ArrayList<>(autoMerge));
+     mergeStatus.put("manualMerge", new ArrayList<>(manualMerge));
+
+     return null;
+ });
+ } catch (Exception e) {
+     throw new RuntimeException(e);
+ }
+
+     return mergeStatus;
+ }
+
+
+}
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/MetadataMergeWorkflowServiceImpl.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/MetadataMergeWorkflowServiceImpl.java
index eeb061d..01092b7 100644
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/MetadataMergeWorkflowServiceImpl.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/services/impl/MetadataMergeWorkflowServiceImpl.java
@@ -1,5 +1,6 @@
package com.adobe.aem.acs.bofa.core.services.impl;

+import com.adobe.aem.acs.bofa.core.constants.AssetType;
import com.adobe.aem.acs.bofa.core.constants.BofaSyncConstants;
import com.adobe.aem.acs.bofa.core.models.MetadataMerge;
import com.adobe.aem.acs.bofa.core.models.MetadataMergeMetadata;
@@ -17,6 +18,8 @@ import org.slf4j.LoggerFactory;

import java.util.Objects;

+import static com.adobe.aem.acs.bofa.core.constants.BofaJcrPropertyConstants.*;
+
@Component(service = MetadataMergeWorkflowService.class, immediate = true)
public class MetadataMergeWorkflowServiceImpl implements MetadataMergeWorkflowService {

@@ -39,13 +42,24 @@ public class MetadataMergeWorkflowServiceImpl implements MetadataMergeWorkflowSe

    MetadataMerge metadataMerge = metadataMergeService.getMetadataMerge(resourceResolver,
metadataMergeMetadata);

-    if(metadataMerge.isContentIdentical()){
-        logger.info("Close asset merge for asset: {}", metadataMergeMetadata.getAssetPath());
-        assetJcrContent.put(BofaJcrUtils.getNamespacedProperty("mergeMetadataStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_CLOSED);
-    } else {
-        throw new IllegalStateException("Cannot close Metadata merge, content is not the same");
-    }
+    logger.info("Close asset merge for asset: {}", metadataMergeMetadata.getAssetPath());
+    assetJcrContent.put(BofaJcrUtils.getNamespacedProperty("mergeMetadataStatus"),
BofaSyncConstants.BOFA_SYNC_CONTENT_FRAGMENT_MERGE_CLOSED);
+
+    String mergeJcrContentPath = BofaSyncConstants.BOFA_SYNC_MERGE_PATH + "/" +

```

```

JcrConstants.JCR_CONTENT;
+   ModifiableValueMap mergeProperties =
Objects.requireNonNull(resourceResolver.getResource(mergeJcrContentPath)).adaptTo(ModifiableValueMap.class);
+
+   BofaJcrUtils.removePathFromProperty(mergeProperties, metadataMergeMetadata.getAssetPath(),
PROP_ASSET_METADATA_PATHS);
+
+   if (metadataMergeMetadata.getAssetType() == AssetType.ASSET){
+       if (!BofaJcrUtils.isAssetInUse(mergeProperties, metadataMergeMetadata.getAssetPath())){
+           BofaJcrUtils.removeNode(resourceResolver, mergeProperties, metadataMergeMetadata.getAssetPath(),
metadataMergeMetadata.getAssetTempPath());
+       }
+   } else if (metadataMergeMetadata.getAssetType() == AssetType.CONTENT_FRAGMENT) {
+       if (!BofaJcrUtils.isContentFragmentInUse(mergeProperties, metadataMergeMetadata.getAssetPath())){
+           BofaJcrUtils.removeNode(resourceResolver, mergeProperties, metadataMergeMetadata.getAssetPath(),
metadataMergeMetadata.getAssetTempPath());
+       }
+   }
+   }
}

```

```

diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/CloseContentMergeServlet.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/CloseContentMergeServlet.java
index b29a8cb..808f67a 100644

```

```

--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/CloseContentMergeServlet.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/CloseContentMergeServlet.java
@@ -37,10 +37,14 @@ public class CloseContentMergeServlet extends SlingSafeMethodsServlet {

    try {
        executor.runWithWriteAccess((ResourceResolver resourceResolver) -> {
-            contentFragmentMergeWorkflowService.closeContentFragmentMerge(resourceResolver,
contentFragmentPath, variationName);
+            contentFragmentMergeWorkflowService.closeContentFragmentVariationMerge(resourceResolver,
contentFragmentPath, variationName);
            return null;
        });
    } catch (Exception e) {
+        response.setStatus(500);
+        response.setContentType("application/json");
+        response.setCharacterEncoding("UTF-8");
+        response.getWriter().write("{\"success\": false}");
        throw new RuntimeException(e);
    }
}

```

```

diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/MergeStatusServlet.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/MergeStatusServlet.java
new file mode 100644
index 0000000..4c71a2a

```

```

--- /dev/null
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/MergeStatusServlet.java
@@ -0,0 +1,56 @@
+package com.adobe.aem.acs.bofa.core.servlets;
+
+import com.adobe.aem.acs.bofa.core.services.MergeStatusService;

```

```

+import com.google.gson.Gson;
+import org.apache.sling.api.SlingHttpServletRequest;
+import org.apache.sling.api.SlingHttpServletResponse;
+import org.apache.sling.api.resource.ResourceResolver;
+import org.apache.sling.api.servlets.HttpConstants;
+import org.apache.sling.api.servlets.SlingSafeMethodsServlet;
+import org.osgi.framework.Constants;
+import org.osgi.service.component.annotations.Component;
+import org.osgi.service.component.annotations.Reference;
+import org.slf4j.Logger;
+import org.slf4j.LoggerFactory;
+
+import javax.servlet.Servlet;
+import java.io.IOException;
+import java.util.List;
+import java.util.Map;
+
+@Component(service = Servlet.class, property = {Constants.SERVICE_DESCRIPTION + "= Merge status servlet",
+    "sling.servlet.paths=" + "/bin/mergeStatus", "sling.servlet.methods=" + HttpConstants.METHOD_GET
+})
+public class MergeStatusServlet extends SlingSafeMethodsServlet {
+
+
+    private static final Logger logger = LoggerFactory.getLogger(MergeStatusServlet.class);
+
+    @Reference
+    private MergeStatusService mergeStatusService;
+
+    @Override
+    protected void doGet(SlingHttpServletRequest request, SlingHttpServletResponse response) throws IOException {
+        logger.info("Getting merge status");
+
+        Gson gson = new Gson();
+
+        try {
+            Map<String, List<String>> mergeStatus = mergeStatusService.getMergeStatus();
+
+            gson.toJson(mergeStatus, response.getWriter());
+
+        } catch (Exception e) {
+            response.setStatus(500);
+            response.setContentType("application/json");
+            response.setCharacterEncoding("UTF-8");
+            response.getWriter().write("{\"success\": false}");
+            throw new RuntimeException(e);
+        }
+
+        response.setContentType("application/json");
+        response.setCharacterEncoding("UTF-8");
+    }
+}
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/EndMergeServlet.java

```

```

b/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/UpdateAssetServlet.java
similarity index 56%
rename from core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/EndMergeServlet.java
rename to core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/UpdateAssetServlet.java
index a0254c8..6931b68 100644
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/EndMergeServlet.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/servlets/UpdateAssetServlet.java
@@ -1,7 +1,8 @@
package com.adobe.aem.acs.bofa.core.servlets;

import com.adobe.aem.acs.bofa.core.executor.JcrResolverExecutor;
-import com.adobe.aem.acs.bofa.core.services.ContentFragmentMergeWorkflowService;
+import com.adobe.aem.acs.bofa.core.services.AssetMergeWorkflowService;
+import com.adobe.aem.acs.bofa.core.services.AssetService;
import org.apache.sling.api.SlingHttpServletRequest;
import org.apache.sling.api.SlingHttpServletResponse;
import org.apache.sling.api.resource.ResourceResolver;
@@ -16,39 +17,48 @@ import org.slf4j.LoggerFactory;
import javax.servlet.ServletException;
import java.io.IOException;

-@Component(service = Servlet.class, property = {Constants.SERVICE_DESCRIPTION + "= End Merge Workflow",
-    "sling.servlet.paths=" + "/bin/endMerge", "sling.servlet.methods=" + HttpConstants.METHOD_GET
+@Component(service = Servlet.class, property = {Constants.SERVICE_DESCRIPTION + "= Update Asset Servlet",
+    "sling.servlet.paths=" + "/bin/updateAsset", "sling.servlet.methods=" + HttpConstants.METHOD_GET
    })
+public class EndMergeServlet extends SlingSafeMethodsServlet {
+public class UpdateAssetServlet extends SlingSafeMethodsServlet {

-    private static final Logger logger = LoggerFactory.getLogger(EndMergeServlet.class);
+    private static final Logger logger = LoggerFactory.getLogger(CloseMetadataMergeServlet.class);

    @Reference
-    JcrResolverExecutor executor;
+    private JcrResolverExecutor executor;

    @Reference
-    private ContentFragmentMergeWorkflowService contentFragmentMergeWorkflowService;
+    private AssetService assetService;
+
+    @Reference
+    private AssetMergeWorkflowService assetMergeWorkflowService;

    @Override
    protected void doGet(SlingHttpServletRequest request, SlingHttpServletResponse response) throws IOException {
-        logger.info("End Merge Workflow");
+        String assetPath = request.getParameter("assetPath");
+        logger.info("Updating asset: {}", assetPath);

        try {
            executor.runWithWriteAccess((ResourceResolver resourceResolver) -> {
-                contentFragmentMergeWorkflowService.endContentFragmentMerge(resourceResolver);
-
+                assetService.updateAssetFromSource(resourceResolver, assetPath);
            }
        }
    }

```

```

+         assetMergeWorkflowService.closeAssetMerge(resourceResolver, assetPath);
+         return null;
+     });
+ } catch (Exception e) {
+     response.setStatus(500);
+     response.setContentType("application/json");
+     response.setCharacterEncoding("UTF-8");
+     response.getWriter().write("{\"success\": false}");
+
+     throw new RuntimeException(e);
+ }
-
-
+     response.setContentType("application/json");
+     response.setCharacterEncoding("UTF-8");
+     response.getWriter().write("{\"success\": true}");
+
+ }
+
+ }
diff --git a/core/src/main/java/com/adobe/aem/acs/bofa/core/util/BofaJcrUtils.java
b/core/src/main/java/com/adobe/aem/acs/bofa/core/util/BofaJcrUtils.java
index 7ca82fc..2559758 100644
--- a/core/src/main/java/com/adobe/aem/acs/bofa/core/util/BofaJcrUtils.java
+++ b/core/src/main/java/com/adobe/aem/acs/bofa/core/util/BofaJcrUtils.java
@@ -1,11 +1,126 @@
package com.adobe.aem.acs.bofa.core.util;

import com.adobe.aem.acs.bofa.core.constants.BofaSyncConstants;
+import com.adobe.aem.acs.bofa.core.models.ContentFragmentInfo;
+import org.apache.commons.lang3.StringUtils;
+import org.apache.sling.api.resource.ModifiableValueMap;
+import org.apache.sling.api.resource.Resource;
+import org.apache.sling.api.resource.ResourceResolver;
+import org.slf4j.Logger;
+import org.slf4j.LoggerFactory;
+
+import javax.jcr.Node;
+import javax.jcr.RepositoryException;
+import java.util.*;

+import static com.adobe.aem.acs.bofa.core.constants.BofaJcrPropertyConstants.*;

public class BofaJcrUtils {

+
+     private static final Logger logger = LoggerFactory.getLogger(BofaJcrUtils.class);
+
+     public static String getNamespacedProperty(String property) {
+         return String.format("%s:%s", BofaSyncConstants.BOFA_JCR_NAMESPACE, property);
+     }
}

```

```

+
+ public static void addPathToProperty(ModifiableValueMap mergeProperties, String path, String property) {
+     Set<String> closedContentFragmentPaths = BofaJcrUtils.getPathSet(mergeProperties, property);
+     closedContentFragmentPaths.add(path);
+     List<String> updatedClosedContentFragmentPaths = new ArrayList<>(closedContentFragmentPaths);
+     mergeProperties.put(BofaJcrUtils.getNamespacedProperty(property),
updatedClosedContentFragmentPaths.toArray(new String[0]));
+ }
+
+
+
+ public static void removePathFromProperty(ModifiableValueMap mergeProperties, String path, String property) {
+     logger.info("Removing path {} from property {}", path, property);
+     Set<String> contentFragmentPaths = BofaJcrUtils.getPathSet(mergeProperties, property);
+     contentFragmentPaths.remove(path);
+     List<String> updatedContentFragmentPaths = new ArrayList<>(contentFragmentPaths);
+     mergeProperties.put(BofaJcrUtils.getNamespacedProperty(property), updatedContentFragmentPaths.toArray(new
String[0]));
+ }
+
+
+
+ public static Set<String> getPathSet(ModifiableValueMap properties, String property) {
+
+     // get the list of existing content fragment paths stored in PROP_CONTENT_FRAGMENT_PATHS
+     List<String> current = Arrays.asList(properties.get(BofaJcrUtils.getNamespacedProperty(property), new
String[0]));
+     logger.info("Existing getPathSet paths: {}", StringUtils.join(current, ","));
+
+     Set<String> existingContentPathSet = new HashSet<>(current);
+ //     existingContentPathSet.addAll(incomingContentFragmentPaths);
+     return existingContentPathSet;
+
+ }
+
+
+
+ public static boolean isAssetInUse(ModifiableValueMap mergeProperties, String contentFragmentPath, String
excludeProperty) {
+     List<String> propertyPaths = new ArrayList<>(List.of(PROP_ASSET_PATHS,
PROP_ASSET_METADATA_PATHS));
+     propertyPaths.remove(excludeProperty);
+     return isPathInUse(mergeProperties, propertyPaths, contentFragmentPath);
+ }
+
+
+
+ public static boolean isAssetInUse(ModifiableValueMap mergeProperties, String contentFragmentPath) {
+     List<String> propertyPaths = List.of(PROP_ASSET_PATHS, PROP_ASSET_METADATA_PATHS);
+     return isPathInUse(mergeProperties, propertyPaths, contentFragmentPath);
+ }
+
+
+
+ public static boolean isContentFragmentInUse(ModifiableValueMap mergeProperties, String contentFragmentPath)
{
+     List<String> propertyPaths = List.of(PROP_CONTENT_FRAGMENT_PATHS,
PROP_ASSET_METADATA_PATHS);

```

```

+     return isPathInUse(mergeProperties, propertyPaths, contentFragmentPath);
+ }
+
+
+ public static boolean isContentFragmentInUse(ModifiableValueMap mergeProperties, String contentFragmentPath,
String excludeProperty) {
+     List<String> propertyPaths = new ArrayList<>(List.of(PROP_CONTENT_FRAGMENT_PATHS,
PROP_ASSET_METADATA_PATHS));
+     propertyPaths.remove(excludeProperty);
+     return isPathInUse(mergeProperties, propertyPaths, contentFragmentPath);
+ }
+
+
+ private static boolean isPathInUse(ModifiableValueMap mergeProperties, List<String> propertyPaths, String path)
{
+     boolean isInUse = false;
+     for (String propertyPath : propertyPaths) {
+         Set<String> set = getPathSet(mergeProperties, propertyPath);
+         if (set.contains(path)) {
+             isInUse = true;
+         }
+     }
+
+     return isInUse;
+ }
+
+
+ public static void removeNode(ResourceResolver resourceResolver, ModifiableValueMap mergeProperties, String
assetPath, String assetTempPath) {
+     logger.info("Attempting to remove node: {}", assetTempPath);
+
+     boolean canDelete = true;
+     List<String> propertyPaths = List.of(PROP_ASSET_PATHS, PROP_CONTENT_FRAGMENT_PATHS,
PROP_CONTENT_FRAGMENT_MISSING_MODEL_PATHS, PROP_ASSET_METADATA_PATHS);
+     for (String propertyPath : propertyPaths) {
+         Set<String> set = getPathSet(mergeProperties, propertyPath);
+         if (set.contains(assetPath)) {
+             canDelete = false;
+         }
+     }
+
+     if(canDelete) {
+         Resource resource = resourceResolver.getResource(assetTempPath);
+         if (resource != null) {
+             try {
+                 Node node = resource.adaptTo(Node.class);
+                 Objects.requireNonNull(node).remove();
+             } catch (RepositoryException e) {
+                 throw new RuntimeException(e);
+             }
+         }
+     } else {
+         logger.warn("Cannot delete path, check merge properties: {}", assetPath);

```

```

+     }
+
+   }
+
+ }
diff --git a/pom.xml b/pom.xml
index 84e5259..b154c30 100644
--- a/pom.xml
+++ b/pom.xml
@@ -39,7 +39,7 @@
</modules>

<properties>
-   <revision>1.0.10-SNAPSHOT</revision>
+   <revision>1.0.11-SNAPSHOT</revision>
   <aem.host>localhost</aem.host>
   <aem.port>4502</aem.port>
   <aem.publish.host>localhost</aem.publish.host>
diff --git a/ui.apps/src/main/content/jcr_root/apps/bofa-sync/components/asset-diff/asset-diff.html
b/ui.apps/src/main/content/jcr_root/apps/bofa-sync/components/asset-diff/asset-diff.html
index 8bb8038..4968d7f 100644
--- a/ui.apps/src/main/content/jcr_root/apps/bofa-sync/components/asset-diff/asset-diff.html
+++ b/ui.apps/src/main/content/jcr_root/apps/bofa-sync/components/asset-diff/asset-diff.html
@@ -22,7 +22,13 @@
<strong>DAM Folder</strong>: <a href="/assets.html${item.assetMergeMetadata.assetParentPath}"
target="_blank">${item.assetMergeMetadata.assetParentPath}</a>
</div>
<sly data-sly-test="${item.mergeStatus == 'open'}">
-   <div data-sly-test="${item.inputStreamsEqual}">
+   <div>
+       <strong>Take from Source</strong>: <a href="/bin/updateAsset?
assetPath=${item.assetMergeMetadata.assetPath}">Take from Source</a>
+   </div>
+   <div>
+       <strong>Keep Destination</strong>: <a href="/bin/closeAssetMerge?
assetPath=${item.assetMergeMetadata.assetPath}">Keep Destination</a>
+   </div>
+   <div>
+       <strong>Close Merge</strong>: <a href="/bin/closeAssetMerge?
assetPath=${item.assetMergeMetadata.assetPath}">Close Merge</a>
+   </div>
</sly>
@@ -39,6 +45,17 @@
</tr>
</thead>
<tbody is="coral-table-body" divider="cell">
+   <tr is="coral-table-row">
+       <td is="coral-table-cell">Asset Details</td>
+       <td is="coral-table-cell">
+           <a href="/assetdetails.html${item.assetMergeMetadata.assetTempPath}"
target="_blank">Source</a>
+       </td>
+       <td is="coral-table-cell">
+           <a href="/assetdetails.html${item.assetMergeMetadata.assetPath}"

```


</sly>
</div>

diff --git a/ui.apps/src/main/content/jcr_root/apps/bofa-sync/components/metadata-diff/metadata-diff.html
b/ui.apps/src/main/content/jcr_root/apps/bofa-sync/components/metadata-diff/metadata-diff.html
index dca0121..a3f6140 100644
--- a/ui.apps/src/main/content/jcr_root/apps/bofa-sync/components/metadata-diff/metadata-diff.html
+++ b/ui.apps/src/main/content/jcr_root/apps/bofa-sync/components/metadata-diff/metadata-diff.html
@@ -26,7 +26,8 @@

</div>
<sly data-sly-test="{item.mergeStatus == 'open'}">
- <div data-sly-test="{item.contentIdentical}">
+<!-- <div data-sly-test="{item.contentIdentical}">-->
+ <div>
 Close Merge: <a href="/bin/closeMetadataMerge?
assetPath={item.metadataMergeMetadata.assetPath}">Close Merge
 </div>
</sly>